

DP4: Control Design for a Drone Race

Maxwell Lutter* and Landon Lopez†

University of Illinois Urbana-Champaign, Urbana, IL, 61801

This paper presents the design and evaluation of feedback control and state estimation for a quadrotor drone tasked with completing a three-dimensional obstacle race. The quadrotor dynamics are linearized about a hovering equilibrium and represented in state-space form. A linear quadratic regulator (LQR) is used to synthesize a full-state feedback controller, while an observer is designed via an LQR-based formulation to provide real-time state estimation from noisy position measurements. Reference tracking is achieved through a guidance law that incrementally updates the equilibrium point toward successive checkpoints, while obstacle avoidance is incorporated using attractive and repulsive potential fields. The controller and observer are implemented and tested in a PyBullet simulation environment featuring randomized initial conditions and obstacle placements. Performance is evaluated over 100 independent trials using metrics of course completion rate and completion time. The proposed approach achieves a 95% success rate and an average completion time of 73.35 s, exceeding the required robustness and speed criteria. Results demonstrate that linear control and estimation methods, when combined with suitable guidance logic, can reliably support agile quadrotor navigation in cluttered environments.

Nomenclature

A, B, C	=	state, input, and output matrices
e	=	state estimation error
f_z	=	net thrust force along body z axis (N)
K, L	=	state-feedback and observer gain matrices (dimensionless)
m, n	=	nonlinear state and input vectors
m_e, n_e	=	equilibrium state and input
p_x, p_y, p_z	=	position in inertial frame (m)
p_{des}	=	desired position from guidance law (m)
$p_{\text{obst},i}$	=	position of obstacle i (m)
Q, R	=	LQR weighting matrices for controller design
Q_o, R_o	=	LQR weighting matrices for observer design
r_{drone}	=	effective drone safety radius (m)
u	=	control input (N and N·m)
v_x, v_y, v_z	=	linear velocity in body frame (m/s)
W, O	=	controllability and observability matrices
x	=	deviation state
$\hat{x}$	=	estimated state
y	=	measurement/output vector (m)
ϕ, θ, ψ	=	roll, pitch, yaw angles (rad)
$\omega_x, \omega_y, \omega_z$	=	angular rates in body frame (rad/s)

I. Introduction

The goal of this project is to develop a quadrotor controller and observer with reference tracking that can reliably complete a three-dimensional obstacle course in a fast and repeatable manner. The course consists of fixed checkpoint

*Undergraduate Student, Department of Aerospace Engineering, Grainger College of Engineering, University of Illinois Urbana-Champaign,

†Undergraduate Student, Department of Aerospace Engineering, Grainger College of Engineering, University of Illinois Urbana-Champaign,

rings at randomized heights and randomly placed obstacles between these rings. The drone must remain airborne, avoid both static obstacles and other drones, and traverse all checkpoints within strict limits on computation time and failure rate. Recent research on agile quadrotor flight shows that safe, high-speed operation in cluttered environments requires close integration of localization, collision avoidance, and feedback control. Yang et al. [1] develop and experimentally validate a multi-quadrotor framework that guarantees collision avoidance while several vehicles cooperatively surround moving ground targets, demonstrating that real-time control under strict safety constraints is feasible on embedded hardware. Related work in future Advanced Air Mobility (AAM) extends these ideas to dense urban airspace: Ballin et al. [2] flight-test a flight-path management system that continuously replans four-dimensional trajectories for many vehicles to maintain separation, avoid conflicts, and satisfy tight scheduling constraints. Both examples treat guidance, constraint handling, and collision avoidance as a coupled control problem, which directly motivates the present project. Our quadrotor race controller can be viewed as a simplified testbed for these broader challenges, in which we design and evaluate guidance and feedback strategies for fast, safe flight through a cluttered environment populated by other vehicles.

II. Theory

A. Creating the State-Space Model

Given the equations of motion that govern the system,*

$$\begin{bmatrix} \dot{p}_x & \dot{p}_y & \dot{p}_z & \dot{v}_x & \dot{v}_y & \dot{v}_z & \dot{\psi} & \dot{\theta} & \dot{\phi} & \dot{\omega}_x & \dot{\omega}_y & \dot{\omega}_z \end{bmatrix}^T = f(m, n), \quad (1)$$

the state and input column vectors m and n are defined as the stack of all known states and nonlinear inputs, respectively:

$$m = \begin{bmatrix} p_x & p_y & p_z & \psi & \theta & \phi & v_x & v_y & v_z & \omega_x & \omega_y & \omega_z \end{bmatrix}^T, \quad n = \begin{bmatrix} \tau_x & \tau_y & \tau_z & f_z \end{bmatrix}^T. \quad (2)$$

By setting the equations of motion f to zero, a viable equilibrium point is obtained. Because f is independent of the drone's position, any position can be chosen to satisfy equilibrium. All other state components are zero at equilibrium, and f_{ze} is set to $0.5 \text{ kg } g$ to counteract gravity. Physically, this corresponds to the drone hovering, stationary and level, at an arbitrary point in space:

$$m_e = \begin{bmatrix} p_{xe} & p_{ye} & p_{ze} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}^T, \quad n_e = \begin{bmatrix} 0 & 0 & 0 & 4.905 \end{bmatrix}^T. \quad (3)$$

The linear state x and input u are defined as

$$x = m - m_e, \quad u = n - n_e. \quad (4)$$

Considering instances near the equilibrium point previously chosen, the nonlinear system can be approximated by the linear state-space and sensor models

$$\dot{x} = Ax + Bu, \quad y = Cx, \quad (5)$$

with A and B calculated by evaluating the Jacobian of the function $f(m, n)$ at the equilibrium values:

$$A = \left. \frac{\partial f}{\partial m} \right|_{m_e, n_e}, \quad B = \left. \frac{\partial f}{\partial n} \right|_{m_e, n_e}. \quad (6)$$

*Due to the size and complexity of the equations of motion, they are not included here, but they can be found in the code accompanying this report.

B. Linearizing the Sensor Model

Given the output of two sensors, one within the center of the “front” rotor and one within the center of the “back” rotor,

$$o(p_x, p_y, p_z, \psi, \theta) = \begin{bmatrix} p_x + \frac{7 \cos(\psi) \cos(\theta)}{40} \\ p_y + \frac{7 \sin(\psi) \cos(\theta)}{40} \\ p_z - \frac{7 \sin(\theta)}{40} \\ p_x - \frac{7 \cos(\psi) \cos(\theta)}{40} \\ p_y - \frac{7 \sin(\psi) \cos(\theta)}{40} \\ p_z + \frac{7 \sin(\theta)}{40} \end{bmatrix}, \quad (7)$$

the C matrix can be calculated similarly by taking the Jacobian of this function $o(p_x, p_y, p_z, \psi, \theta)$ at the same equilibrium values:

$$C = \left. \frac{\partial o}{\partial m} \right|_{m_e, n_e}. \quad (8)$$

C. Controllability and Observability

Now that the A , B , and C matrices have been constructed, it is prudent to first check that the system is controllable (any state can be reached) and observable (sensor data can be used to estimate the full state in a finite amount of time). To do this, the ranks of the controllability matrix

$$W = \begin{bmatrix} B & AB & \dots & A^{n-1}B \end{bmatrix} \quad (9)$$

and the observability matrix[†]

$$O = \begin{bmatrix} C \\ CA \\ \vdots \\ CA^{n-1} \end{bmatrix} \quad (10)$$

are checked against the number of states, in this case 12. In our implementation,

$$\text{rank}(W) = \text{rank}(O) = 12,$$

so the linearized system is both controllable and observable about the chosen equilibrium.

D. Controller Design

A closed-loop feedback design

$$u = -Kx \quad (11)$$

is applied to the state-space model, yielding

$$\dot{x} = (A - BK)x. \quad (12)$$

From the general solution

$$x(t) = e^{(A-BK)t}x(0), \quad (13)$$

the system is stable when the eigenvalues of the matrix $(A - BK)$ possess negative real parts. For a system as complex as this drone, it is important to control the relative weighting between reaching a desired state and the effort required to do so. Consequently, a linear quadratic regulator (LQR) method was applied to compute the gain matrix K .

[†]Matrices as large as A , B , C , W , and O are not included in this report. The full derivation of these matrices and the proof of controllability and observability can be found in the accompanying code.

In the code used to generate the results, the Q and R matrices were initialized as diagonal matrices of sizes 12×12 and 4×4 , respectively. The values of their non-zero elements were adjusted during testing to prioritize stable flight. Using the function `solve_continuous_are` to solve the algebraic Riccati equation for P and defining the K matrix as

$$K = R^{-1} B^T P \quad (14)$$

produces a gain matrix K that minimizes the cost function

$$\int_{t_0}^{t_f} (x^T Q x + u^T R u) dt, \quad (15)$$

where the first term penalizes deviation of the state from the desired equilibrium point and the second term penalizes control effort.

Similarly, for the observer gain matrix L , and noting that the Kalman filter is analogous to the LQR formulation, the matrices Q_o and R_o were initialized as diagonal matrices of sizes 6×6 and 12×12 , respectively. The same `solve_continuous_are` function was used, replacing A and B with A^T and C^T , to obtain P_o , and defining

$$L = P_o C^T Q_o. \quad (16)$$

Diagonal entries of Q_o encode how much measurement noise is expected for each sensor (larger values mean the observer trusts that sensor less), while diagonal entries of R_o encode expected process disturbances in each state (larger values lead to more aggressive correction of that state). As with Q and R , these entries were tuned in simulation to achieve accurate, noise-robust state estimates that work well with the controller.

E. Reference Tracking and Obstacle Avoidance

Due to the presence of sensor noise, an observer must be implemented to estimate the state at each moment. Defining the estimation error $e = x - \hat{x}$, the error dynamics satisfy

$$\dot{e} = (A - LC)e, \quad (17)$$

so the state estimate will converge to the actual state if the eigenvalues of $(A - LC)$ all have negative real parts. Verification of the eigenvalues confirms sufficient observer accuracy. With this established, the observer is implemented within the controller class. The estimated state is denoted $\hat{x}$, and the desired control input is defined as

$$u = -K\hat{x}. \quad (18)$$

The vector $\hat{x}$ is initialized at the initial state of the simulation, and at each time step Δt , the derivative

$$\dot{\hat{x}} = A\hat{x} + Bu - L(C\hat{x} - y) \quad (19)$$

is integrated forward, where the output y is the sensor-derived measurement vector. The commanded inputs τ_x , τ_y , τ_z , and f_z are taken from their respective entries in the input vector u .

Defining the equilibrium position of the drone is also necessary. Although the equilibrium state could be set to the center of each successive ring, a smoother and more reliable path is obtained by defining the equilibrium to trace small steps between the drone and the next ring. Each obstacle is modeled as a sphere of center $p_{\text{obst},i}$. The desired position p_{des} is defined as

$$p_{\text{des}} = \hat{p} + k_{\text{des}}(h_{\text{attract}}(\hat{p}) + h_{\text{repel}}(\hat{p})). \quad (20)$$

The position components of the desired state are set to p_{des} , and the controller uses the deviation between the estimated state and this moving reference in the feedback law. That is, the input is computed as $u = -K(\hat{x} - x_{\text{des}})$, where x_{des} has position entries equal to p_{des} and all other entries zero. The constant k_{des} specifies the step size toward the goal, and h_{attract} and h_{repel} are potential functions that attract the drone toward the next ring and repel it from obstacles. The attractive potential is chosen as

$$h_{\text{attract}}(\hat{p}) = k_{\text{attract}} \frac{p_{\text{ring}} - \hat{p}}{\|p_{\text{ring}} - \hat{p}\|}, \quad (21)$$

and the repulsive potential is

$$h_{\text{repel}}(\hat{p}) = k_{\text{repel}} \sum_i \frac{1}{d_i(\hat{p})^2} \nabla d_i(\hat{p}), \quad (22)$$

where

$$d_i(\hat{p}) = \|\hat{p} - p_{\text{obst},i}\| - r_{\text{drone}}, \quad \nabla d_i(\hat{p}) = \frac{\hat{p} - p_{\text{obst},i}}{\|\hat{p} - p_{\text{obst},i}\|}. \quad (23)$$

The obstacle position $p_{\text{obst},i}$ is defined from the center of obstacle i , and r_{drone} is an effective safety radius from the center of the drone. As the drone approaches an obstacle, $d_i(\hat{p})$ decreases and the repulsive term grows like $1/d_i(\hat{p})^2$, producing a steering direction that pushes the desired position away from nearby obstacles while the attractive term pulls it toward the next ring.

III. Experimental Methods

Testing was done within the PyBullet virtual environment, which simulated the full physics of the drone, inputs, and collision with obstacles. Over the course of 100 tests, where the initial position within the starting ring was randomized to ensure a robust statistical analysis, the sensors measured the locations of the “front” and “back” rotors, while the controller generated control torques within the rotors based on the feedback law defined by the matrices K and L . Each simulation ran until the drone completed the course, flew out of bounds, or was deemed inactive. Each trial logged the states and state estimates of the position, velocity, attitude angles, and angular velocities. The results of each trial, whether the drone finished and how long it took, were also recorded.

As the purpose of the report is to create a drone controller that is both robust enough to navigate a complex layout of goals and obstacles and to do so quickly, the two requirements are concerned with success rate and completion time. The first requirement made of the drone was that it shall achieve a course completion rate of 80%. This was verified by running 100 trials and tracking each trial where the `get_results` function of the simulator returned a successful completion. The number of entries within that list was compared to the number of trials to obtain the success rate percentage. The second requirement made of the drone was that, of the number of times it completed the course, it shall achieve an average completion time of less than 80 s. To verify this, the final completion time for each trial where the drone completed the course was averaged over the number of completions.

IV. Results and Discussion

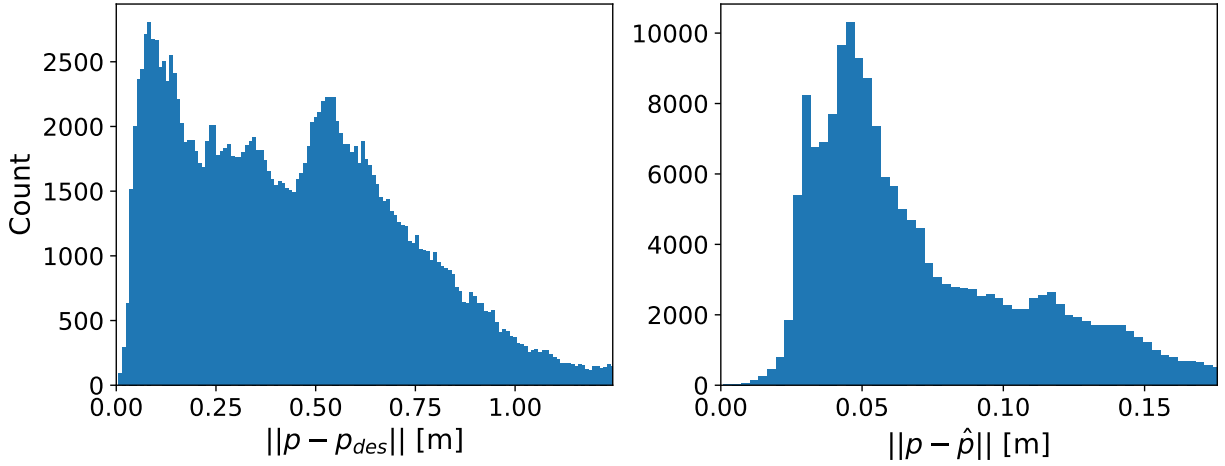


Fig. 1 These histograms of the state errors and state-estimation errors show that they remained close to zero throughout the simulation except in failure cases.

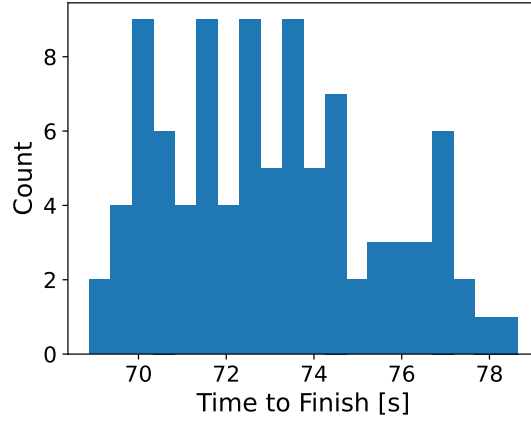


Fig. 2 All of the successful runs finished below the requirement of 80 s. The average completion time of all the finishing runs was 73.35 s.

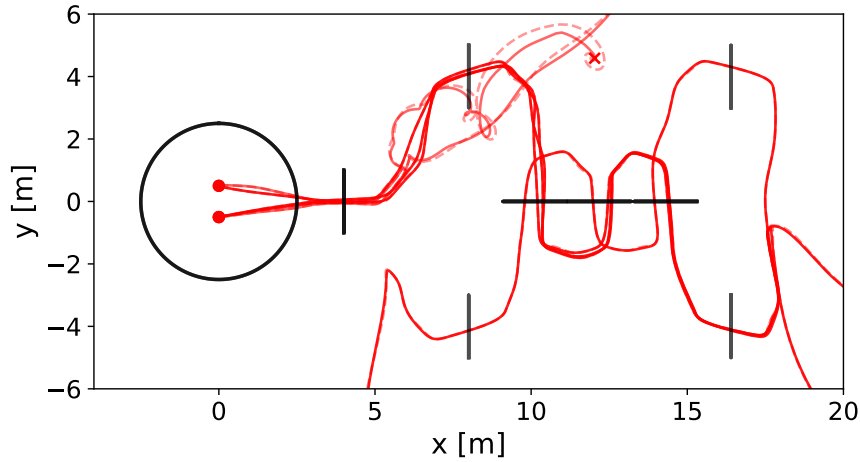


Fig. 3 Overhead view of the trajectories from the five failed trials. All failures occurred near the region of the course where obstacles are spawned, indicating that difficulty stabilizing during or following flight through these obstacles led to the few incomplete trials.

A. State Observer

As can be seen from the central graphs within Figs. 1 and 2, the state-estimation error remains near zero throughout the majority of time steps. As can be seen in the trajectory plot in Fig. 3, deviations in state-estimation error from the actual state follow directly from sharp changes in direction, indicating collisions. One source of disparity between the state and the desired state comes from the transition from one desired position to the next.

B. Success Rate

Following the simulation of 100 trials, the drone was able to complete the course 95% of the time. This success rate, far higher than the required 80%, demonstrates the robust nature of the drone controller to navigate through and around the obstacles in the course. That said, it does not seem unreasonable to believe that a success rate of 100% is achievable. Placing greater focus on obstacle avoidance, particularly increasing the distance the drone flies from the ring's edge, and giving higher weights to maintaining mostly level flight might bridge this last 5% gap.

C. Average Completion Time

Of the 95 successful course completions, the drone managed to beat the required time of 80 s in all 95, with an average completion time of 73.35 s. While the first half of this result is certainly encouraging, the second half contains the main takeaway. There is a clear indication that the original requirement overestimated the time required to complete the course. As can be seen in Fig. 2, the fastest completion time only took around 68.36 s. The main design philosophy of the controller was to prioritize the consistency of the drone and, although both requirements have been met, some sacrifices to speed were made.

V. Conclusion

Working toward the ultimate goal of this paper, the creation of a drone that is both consistent and fast in its completion of the obstacle course, the equations of motion that govern the dynamics of a quadcopter were linearized around a level hovering attitude at an arbitrary point in space. The state-space model was created and the K and L gain matrices were derived using a linear quadratic regulator. The implementation of reference tracking and collision avoidance ensured a more robust and consistent drone controller. Over the course of 100 trials, the controller managed to complete the course 95 times with an average completion time of 73.35 s. From the overall low state and state-estimation errors, the implemented observer managed to approximate the true state accurately throughout the testing, failing only at the busiest sections and closest approaches. It is the opinion of the authors that there are many improvements that could still be made. First and foremost, there is room for further tuning of the weights of the Q and R matrices for both the controller and observer. Secondly, it is recommended that smoother trajectory tracking be pursued to reduce the sudden changes in the desired position. Finally, testing at higher speeds appears to be viable considering the already consistent completion of the course.

VI. Appendix

A. Project Contributions

This project was largely collaborative, with shared contributions on each component. Maxwell Lutter contributed to linearizing the equations of motion and sensor model, strategic decisions about drone navigation, controller and observer gain selection, automated data collection from repeated simulations, and report revisions and edits for content and formatting, especially for the theory section. Landon Lopez contributed to controller implementation in the code, obstacle avoidance, and writing much of the report, especially the experimental methods and the structure of the theory section. The report was written both collaboratively and separately, with writing and revisions on all sections from each member. Maxwell was the primary developer of the final drone code, along with figure generation, and wrote the introduction. Landon primarily contributed to the analysis and conclusion sections and wrote the abstract.

References

- [1] Yang, G., Qi, J., Wang, M., Peng, Y., Wu, C., Ping, Y., and Huang, H., “Localization and tracking of multiple quadrotors with collision avoidance: Theory and experiment,” *Defence Technology*, Vol. 50, 2025/08/, pp. 338 – 50. URL <http://dx.doi.org/10.1016/j.dt.2025.04.002>.
- [2] Ballin, M. G., Barrows, B. A., Nelson, S. L., Underwood, M. C., Fettrow, T. D., Wing, D. J., and Karr, D. A., *Flight Evaluation of a Flight Path Management System for High Density Advanced Air Mobility*, ??? <https://doi.org/10.2514/6.2024-3645>, URL <https://arc.aiaa.org/doi/abs/10.2514/6.2024-3645>.